EventSure - Event Insurance Web Application

System Overview Document

---

1. Project Objective

The objective of this project is to design and develop a secure, scalable, and workflow-driven Event Insurance Web Application that digitizes the complete lifecycle of event insurance — from policy application to claim settlement.

The system aims to replace manual processing with a structured, role-based digital platform that ensures:

- Secure authentication and authorization
- Transparent approval workflows
- Dynamic premium calculation based on risk assessment
- Structured claims processing
- Financial tracking and reporting
- Dashboard-driven analytics

The application focuses specifically on Event Insurance products such as weddings, concerts, corporate events, sports tournaments, and exhibitions.

---

2. Technology Stack

Backend

- ASP.NET Core Web API (.NET 8)
- Entity Framework Core (ORM)
- LINQ for querying
- JWT Authentication
- Role-Based Authorization
- Global Exception Handling Middleware
- Logging Middleware
- xUnit for Unit Testing
- Swagger for API Documentation

Frontend

- Angular (Latest Version)
- TypeScript
- Reactive Forms
- Angular Routing
- Route Guards
- HTTP Interceptors (JWT token attachment)
- Angular Material / Tailwind CSS

Database

- SQL Server
- Primary Key / Foreign Key Constraints
- Indexed Columns for performance
- GROUP BY and CTE-based analytical queries

---

3. User Roles

The system supports four primary roles:

3.1 Admin

- Manage users and roles
- Approve policy applications
- Monitor dashboards and analytics
- Manage policy products
- View reports and revenue metrics

3.2 Agent

- Assist customers in policy creation
- Review submitted applications
- Forward applications for approval
- Track commissions

3.3 Customer

- Register and login
- Apply for event insurance
- Upload documents
- Make premium payments

- Raise claims
- Track claim status

3.4 Claims Officer

- Review submitted claims
- View fraud risk indicator
- Approve or reject claims
- Initiate settlement
- Track settlement timelines

---

4. Core Functional Modules & Workflow

4.1 User & Security Module

Features:

- User Registration & Login
- JWT-based Authentication
- Role-Based Authorization
- Password hashing
- Account lock after failed attempts
- Secure API endpoints
- Angular Route Guards
- JWT HTTP Interceptor

Workflow:

User → Login → Role verified → Redirect to role-specific dashboard

4.2 Dashboard Module

Each role has a customized dashboard displaying metrics and graphical analytics.

Admin Dashboard:

- Total active policies
- Pending approvals
- Monthly premium revenue
- Claims approval rate
- Risk category distribution

Agent Dashboard:

- Assigned customers
- Policies processed
- Commission earned
- Pending applications

Customer Dashboard:

- Active policies
- Payment history
- Upcoming premium dues
- Claims raised

Claims Officer Dashboard:

- Assigned claims
- Claims under review
- Fraud-flagged claims
- Average settlement time

4.3 Event Policy Application Module

This module handles the complete policy application lifecycle.

Step 1 – Policy Selection

Customer selects Event Insurance.

Step 2 – Event Details Submission

Customer provides:

- Event Type
- Event Date & Duration
- Event Location
- Expected Attendees
- Event Budget
- Coverage Required
- Special Risk Factors (Outdoor venue, Fireworks, VIP presence, etc.)

4.4 Risk & Premium Engine Module

This module calculates risk score and final premium dynamically.

Risk Scoring Logic (Rule-Based)

Risk score is calculated based on:

- Event Budget
- Attendee count
- Location type
- Special risk factors

Outputs:

- RiskScore
- RiskCategory (Low / Medium / High)

Premium Calculation:

BasePremium = CoverageAmount × BaseRate
FinalPremium = BasePremium × RiskMultiplier

This makes premium dynamic and risk-based.

4.5 Policy Approval Workflow

Customer → Submit Application
Agent → Review & Forward
Admin → Approve/Reject
If Approved → Active Policy Generated

Policy Lifecycle:

Draft → Submitted → Forwarded → Approved → Active → Expired/Cancelled

4.6 Payment & Commission Module

Features:

- Premium invoice generation
- Installment (EMI) tracking
- Due date monitoring
- Payment history
- Agent commission auto-generation
- Commission payout tracking

4.7 Claims Management Module

Claim Workflow:

Customer → Raise Claim
System → Fraud Risk Check
Claims Officer → Review
Approve/Reject
Settlement Processing

Claim Lifecycle:

Submitted → UnderReview → Approved/Rejected → Settled

Fraud Detection (Rule-Based):

FraudFlag triggered if:

- Claim amount > 80% of coverage
- Policy active for very short duration
- High-risk category policy

4.8 Notification Module

Notifications generated for:

- Policy submission
- Policy approval/rejection
- Payment due reminder
- Claim status updates
- Policy expiry reminder

Stored in database and displayed in dashboard.

4.9 Reporting & Analytics Module

Reports Include:

- Policies by event type
- Monthly premium revenue
- Claims by status
- Risk distribution
- Agent performance

Technical Implementation:

- GROUP BY queries

- CTE-based reports
- Server-side pagination
- Export to CSV

4.10 Audit Logging Module

Tracks:

- Insert operations
- Update operations
- Delete operations
- User performing action
- Timestamp
- Previous & new values

Ensures accountability and traceability.

---

5. End-to-End System Workflow

Policy Application Flow

Customer
→ Select Event Insurance
→ Provide Event Details
→ Risk & Premium Calculation
→ Submit Application
→ Agent Review
→ Admin Approval
→ Active Policy Creation
→ Payment
→ Policy Active

Claims Processing Flow

Customer
→ Raise Claim
→ Fraud Check
→ Claims Officer Review
→ Approval / Rejection
→ Settlement
→ Status Update

---

6. Database Tables Required

Below are the core tables required for full implementation:

Identity & Security

- Users
- Roles
- UserRoles
- RefreshTokens

Policy Management

- PolicyProducts
- PolicyApplications
- EventDetails
- ActivePolicies

Risk & Premium

- RiskAssessments

Payment & Commission

- Payments
- Installments
- AgentCommissions

Claims

- Claims
- ClaimDocuments

Notifications

- Notifications

Reporting & Audit

- AuditLogs

---

7. Non-Functional Requirements

- Clean Layered Architecture
- RESTful API standards
- Server-side pagination
- Optimized SQL queries
- Secure data handling
- Global exception handling
- Middleware logging
- Cloud deployment readiness

---

8. Clean Architecture Structure

Solution Structure:

InsuranceSystem.sln

- Insurance.API
- Insurance.Application
- Insurance.Domain
- Insurance.Infrastructure
- Insurance.Tests

Layer Responsibilities:

Domain → Entities
Application → Business logic & services
Infrastructure → Database & external services
API → Controllers

By Srujana